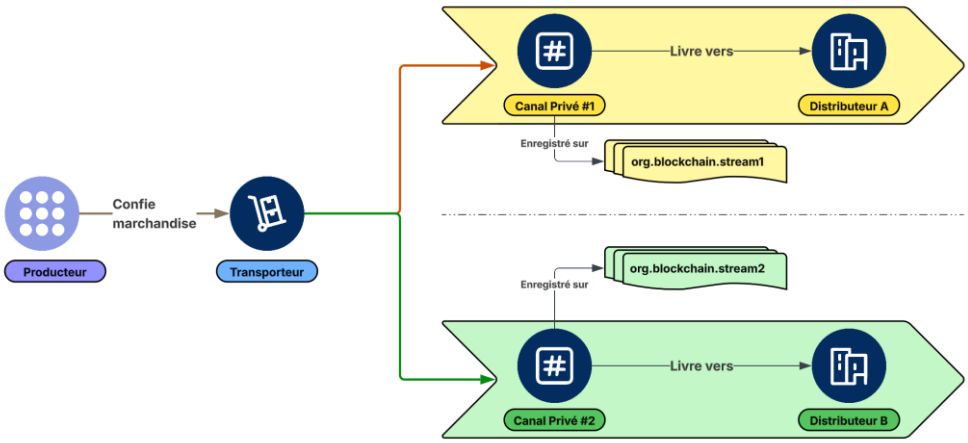
I. Présentation du cas d'étude

Nous utiliserons comme implémentation de la blockchain dans la problématique de traçabilité alimentaire l'outil open-source MultiChain : https://www.multichain.com/, en particulier dans sa version Enterprise (démo gratuite) pour bénéficier des canaux permissionés. MultiChain est une plateforme spécialisée dans la création et le déploiement de blockchains privées ou avec permission pour les entreprises. Elle permet de configurer finement les droits d'accès et la gestion d'actifs sans nécessiter de minage complexe, facilitant l'intégration de la blockchain dans les processus métier. La plateforme n'est pas un fournisseur de cloud de BaaS en tant que tel mais met plutôt à disposition des binaires téléchargeables qui permettent de connecter plusieurs pairs ensembles et d'interagir avec tout l'écosystème de la blockchain locale selon un modèle JSON-RPC.

Notre cas d'étude portera sur la distribution des produits d'un producteur de produits animaliers (lait, viande, œufs...) auprès de ses revendeurs. Le producteur fait ainsi appel à un transporteur qui achemine ses marchandises auprès d'un distributeur A et d'un autre distributeur B. Tous dans une volonté de transparence et de traçabilité prendront part à une même blockchain. Cependant, pour des raisons de confidentialité et de concurrence, les distributeurs ne souhaitent pas afficher publiquement, c'est-à-dire à leurs concurrents, les produits et les quantités qu'ils commandent auprès du producteur.

Chaque participant (producteur, transporteur, distributeurs) possèdera un nœud (les possibilités de mise à l'échelle seront discutées ultérieurement) au sein de la blockchain de l'organisation qui lui permettra d'enregistrer les produits traités. Les distributeurs feront partie de la même organisation mais ne participeront que dans des registres (canaux) séparés privés avec le transporteur et le producteur en excluant l'autre distributeur concurrent pour répondre à la problématique de confidentialité.

Voici un schéma résumant l'organisation interne des participants :



Vous pourrez télécharger le répertoire github à l'adresse :
https://github.com/Nirij3m/foodchain-ledger

Pour initialiser et configurer la blockchain automatiquement, vous pouvez utiliser les commandes suivantes depuis le répertoire du projet :

```
docker compose up --build
# Dans un nouveau terminal, toujours dans le répertoire du projet
chmod +x setup_blockchain.sh
./setup_blockchain
# Puis suivez les instructions du script
```

Si vous souhaitez comprendre les actions réalisées par le script, la section suivante vous permettra de réaliser l'installation et la configuration manuelle de la blockchain.

a. Configuration manuelle de la blockchain

A l'exécution de l'orchestrateur, le nœud producteur créé la blockchain *org_blockchain* et se met en écoute à l'adresse `org_blockchain@[IP_NODE_PROD]:[PORT]` (les modalités de gestion de la blockchain seront discutées ultérieurement) et configure les identifiants RPC avec ceux indiqués dans le fichier `compose.yaml` pour une connexion ultérieure avec l'explorateur web :

```
Attaching to distributeur-a-node-1, distributeur-b-node-1, producteur-node-1, transporteur-node-1
producteur-node-1  |
producteur-node-1  | MultiChain 2.3.3 Utilities (latest protocol 20013)
producteur-node-1  |
producteur-node-1  | Blockchain parameter set was successfully generated.
producteur-node-1  | You can edit it in /root/.multichain/org_blockchain/params.dat before running multichaind for
the first time.
producteur-node-1  |
producteur-node-1  | To generate blockchain please run "multichaind org_blockchain -daemon".
producteur-node-1  | Configuration générée :
producteur-node-1  | rpcuser="uqac"
producteur-node-1  | rpcpassword="foodchainsafety"
producteur-node-1  | Démarrage du service :
producteur-node-1  |
producteur-node-1  | MultiChain 2.3.3 Daemon (Enterprise Demo Edition, latest protocol 20013)
producteur-node-1  |
producteur-node-1  | NOTE! THIS DEMO VERSION CAN BE USED DURING THE FIRST 3 MONTHS OF A BLOCKCHAIN.
producteur-node-1  |
producteur-node-1  | Starting up node...
producteur-node-1  |
producteur-node-1  | Looking for genesis block...
producteur-node-1  | Genesis block found
producteur-node-1  |
producteur-node-1  | Other nodes can connect to this node using:
producteur-node-1  | multichaind org_blockchain@172.24.0.5:6493
producteur-node-1  |
producteur-node-1  | Listening for API requests on port 6492 (local only - see rpcallowip setting)
producteur-node-1  |
producteur-node-1  | Node ready.
producteur-node-1  |
```

Avant de créer des canaux de communications privées, il faut que tous les autres acteurs de la blockchain (le transporteur et les deux distributeurs) demandent à participer à cette dernière. Pour cela il faut se connecter au nœud docker de l'acteur à intégrer et exécuter la commande indiquée précédemment par le nœud producteur :

```
docker ps # Pour lister le nom des conteneurs
docker exec [NOM_IMAGE_CONTENEUR ou ID] multichaind
org_blockchain@[IP_NODE_PROD]:[PORT]
```

Par exemple, pour le nœud du transporteur :

```
nirin@LegIoN:~$ docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS          PORTS
                                   NAMES
f47930ea1a45   nirin-producteur-node          "bash -c 'set -e\nmul…"   12 minutes ago   Up 12 minutes   0.0.0.0:67
90->6790/tcp, [::]:6790->6790/tcp   nirin-producteur-node-1
f9c8f17c7987   nirin-distributeur-b-node      "/bin/bash"              12 minutes ago   Up 12 minutes   0.0.0.0:67
93->6790/tcp, [::]:6793->6790/tcp   nirin-distributeur-b-node-1
2503f2fbe937   nirin-distributeur-a-node      "/bin/bash"              12 minutes ago   Up 12 minutes   0.0.0.0:67
92->6790/tcp, [::]:6792->6790/tcp   nirin-distributeur-a-node-1
5f24519ab948   nirin-transporteur-node        "/bin/bash"              12 minutes ago   Up 12 minutes   0.0.0.0:67
91->6790/tcp, [::]:6791->6790/tcp   nirin-transporteur-node-1
nirin@LegIoN:~$ docker exec nirin-transporteur-node-1 multichaind org_blockchain@172.24.0.4:5769
Retrieving blockchain parameters from the seed node 172.24.0.4:5769 ...
Blockchain successfully initialized.

Please ask blockchain admin or user having activate permission to let you connect and/or transact:
multichain-cli org_blockchain grant 1F19jJfLWaur2oc5UM3bJTDwMK5xAyPzWmJVcV connect
multichain-cli org_blockchain grant 1F19jJfLWaur2oc5UM3bJTDwMK5xAyPzWmJVcV connect,send,receive


MultiChain 2.3.3 Daemon (Community Edition, latest protocol 20013)
```

Le nœud administrateur (producteur) doit ensuite accepter la demande en exécutant la commande surlignée en vert. La chaîne de caractères dans la commande représente l'identifiant MultiChain du nœud, il sera judicieux de le noter dans un coin car il sera réutilisé plus tard :

```
docker exec [NOM_IMAGE_CONTENEUR_PROD ou ID_PROD] multichain-cli
org_blockchain grant [ID_MULTICHAIN_NOEUD] connect
```

```
nirin@LegIoN:~$ docker exec nirin-producteur-node-1 multichain-cli org_blockchain grant 1F19jJfLWaur2oc5UM3bJTDw
MK5xAyPzWmJVcV connect
{"method":"grant","params":["1F19jJfLWaur2oc5UM3bJTDwMK5xAyPzWmJVcV","connect"],"id":"71379988-1776547999","chai
n_name":"org_blockchain"}

7afa127c744bfdbf49135ad4c73b1f12ec40da29f157794090ba9ed2468c621d
```

Il faut ensuite répéter cette opération pour les autres nœuds distributeurs. Pour le distributeur A :

```
nirin@LegIoN:~$ docker exec nirin-distributeur-a-node-1 multichaind org_blockchain@172.24.0.4:5769
Retrieving blockchain parameters from the seed node 172.24.0.4:5769 ...
Blockchain successfully initialized.

Please ask blockchain admin or user having activate permission to let you connect and/or transact:
multichain-cli org_blockchain grant 1LD9v8rdbLdQTLNurMm8P9wsjXrh6BYCs7HW7z connect
multichain-cli org_blockchain grant 1LD9v8rdbLdQTLNurMm8P9wsjXrh6BYCs7HW7z connect,send,receive


MultiChain 2.3.3 Daemon (Community Edition, latest protocol 20013)

nirin@LegIoN:~$ docker exec nirin-producteur-node-1 multichain-cli org_blockchain grant 1LD9v8rdbLdQTLNurMm8P9ws
jXrh6BYCs7HW7z connect
{"method":"grant","params":["1LD9v8rdbLdQTLNurMm8P9wsjXrh6BYCs7HW7z","connect"],"id":"74378869-1776548132","chai
n_name":"org_blockchain"}

50a1f12150619323f154ad88e8a1e2f62fb32774cdf955588f9999659e33160b
nirin@LegIoN:~$
```

Pour le distributeur B :

```
nirin@LegIoN:~$ docker exec nirin-distributeur-b-node-1 multichaind org_blockchain@172.24.0.4:5769
Retrieving blockchain parameters from the seed node 172.24.0.4:5769 ...
Blockchain successfully initialized.

Please ask blockchain admin or user having activate permission to let you connect and/or transact:
multichain-cli org_blockchain grant 13ibdV5hShxA9cKdn651gGvwY2uTgMQ8mGsBVR connect
multichain-cli org_blockchain grant 13ibdV5hShxA9cKdn651gGvwY2uTgMQ8mGsBVR connect,send,receive


MultiChain 2.3.3 Daemon (Community Edition, latest protocol 20013)

nirin@LegIoN:~$ docker exec nirin-producteur-node-1 multichain-cli org_blockchain grant 13ibdV5hShxA9cKdn651gGvw
Y2uTgMQ8mGsBVR connect
{"method":"grant","params":["13ibdV5hShxA9cKdn651gGvwY2uTgMQ8mGsBVR","connect"],"id":"73145366-1776548379","chai
n_name":"org_blockchain"}

d1602493366db0489bb0e889879a1637b6a56da381d5dcde928d84cb234621fd
```

Enfin, il faut reconnecter les nouveaux nœuds (transporteur, distributeur A, distributeur B) à la blockchain à l'aide de la commande suivante :

```
docker exec [NOM_IMAGE_CONTENEUR ou ID] multichaind org_blockchain -daemon
```

Nous allons ensuite nous connecter au nœud administrateur producteur et configurer les différents canaux depuis le CLI du démon multichaind :

```
docker exec -it [NOM_IMAGE_CONTENEUR_PROD ou ID_PROD] bash # Se connecter
en mode interactif au conteneur
multichain-cli org_blockchain # Utilise l'interface CLI de multichain
getinfo # Informations générales sur la blockchain
getpeerinfo # Liste les noeuds qui sont connectés à la blockchain
```

Depuis la CLI de multichaind, nous allons configurer deux canaux (streamA et streamB) avec des restrictions sur les permissions de lecture et d'écriture pour ne permettre qu'aux participants explicitement autorisés à y stocker des transactions et en lire l'historique.

```
create stream streamA '{"restrict":"read,write"}'
create stream streamB '{"restrict":"read,write"}'
```

Le streamA permettra au producteur, transporteur et distributeur A d'enregistrer ensemble leurs transactions successives et le streamB permettra de faire la même chose pour le producteur, transporteur et distributeur B, sans que les distributeurs puissent consulter les transactions de l'autre canal (confidentialité) :

```
org_blockchain: create stream streamA '{"restrict":"read,write"}'
{"method":"create","params":["stream","streamA",{"restrict":"read,write"}],"id":"80963422-1776549758","chain_nam
e":"org_blockchain"}

32d95ce2914bdbb42f27b289c9128fcd7e36a3b113dff2df81dd8cdd0b67b8cb
org_blockchain: create stream streamB '{"restrict":"read,write"}'
{"method":"create","params":["stream","streamB",{"restrict":"read,write"}],"id":"64255118-1776549980","chain_nam
e":"org_blockchain"}

6bee1e8e30af5cc4479e8829bd8c9bc2dcf1a6984ba1f9d859e7e9a6ea445a02
org_blockchain:
```

Pour ce faire, chaque nœud devra se voir attribuer les permissions de lecture et d'écriture dans le canal avant de pouvoir s'y abonner et interagir avec. Depuis le CLI multichain du nœud administrateur producteur il faut donner les permissions avec les commandes suivantes :

```
grant [ID_MULTICHAIN_NOEUD] send # Permission générale préalable
grant [ID_MULTICHAIN_NOEUD] [CANAL].read # Lecture du canal
grant [ID_MULTICHAIN_NOEUD] [CANAL].write # Ecriture dans le canal
```

Puis le nœud nouvellement permissionné peut désormais s'abonner au canal depuis son interface avec la commande :

```
docker exec [NOM_IMAGE_CONTENEUR ou ID] multichain-cli org_blockchain
subscribe [CANAL]
```

Par exemple, pour le nœud transporteur portant comme identifiant *1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM* et ayant besoin d'accéder au streamA et B :

```
org_blockchain: grant 1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM send
{"method":"grant","params":["1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM","send"],"id":"51302553-1776625494","chain_nam
e":"org_blockchain"}

d287cd9bbcc8a723462cf7b600bfd09a6490f4927dfe9917d969756bc99f4acb
org_blockchain: grant 1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM streamA.read
{"method":"grant","params":["1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM","streamA.read"],"id":"33267630-1776625503","c
hain_name":"org_blockchain"}

e3f2a2723be4e5ae198712cba8022e729c93fc3c9ee216a2dc7f1ccac9f3d926
org_blockchain: grant 1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM streamA.write
{"method":"grant","params":["1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM","streamA.write"],"id":"43791025-1776625517","
chain_name":"org_blockchain"}

e79a2dc6acdd6a7773835a39ce981c77e8293715160eaae36be27fe1630ae2a1
org_blockchain: grant 1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM streamB.read
{"method":"grant","params":["1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM","streamB.read"],"id":"15228323-1776625525","c
hain_name":"org_blockchain"}

6117d682bfb5c1154107f6b12a563ae128e7a86c1a963473dcc71b1e8e382f42
org_blockchain: grant 1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM streamB.write
{"method":"grant","params":["1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM","streamB.write"],"id":"93720655-1776625530","
chain_name":"org_blockchain"}

5f3857f4b68d0fa5e048e46227657b071136ddeecad13c0967096be38938bf58
```

```
nirin@LegIoN:~$ docker exec nirin-transporteur-node-1 multichain-cli org_blockchain subscribe streamA
{"method":"subscribe","params":["streamA"],"id":"93758425-1776625807","chain_name":"org_blockchain"}

nirin@LegIoN:~$ docker exec nirin-transporteur-node-1 multichain-cli org_blockchain subscribe streamB
{"method":"subscribe","params":["streamB"],"id":"89849719-1776625813","chain_name":"org_blockchain"}
```

Il faut ensuite configurer les permissions d'accès au producteur (bien qu'il soit admin) et aux distributeurs dans leurs canaux respectifs. Voici un tableau récapitulatif des différentes permissions pour chaque canaux et nœuds :

Noeuds	*streamA*	*streamB*
	read/write	read/write
Producteur	OUI/OUI	OUI/OUI
Transporteur	OUI/OUI	OUI/OUI
Distributeur A	OUI/OUI	NON/NON
Distributeur B	NON/NON	OUI/OUI

Après configuration, nous pouvons vérifier que le distributeur A ne peut effectivement pas s'inscrire au flux du canal (stream) B :

```
nirin@LegIoN:~$ docker exec nirin-distributeur-a-node-1 multichain-cli org_blockchain subscribe streamB
{"method":"subscribe","params":["streamB"],"id":"37799028-1776626594","chain_name":"org_blockchain"}

error code: -704
error message:
This wallet doesn't have keys with read permission for stream streamB
```

Il ne pourra donc pas lire le contenu des transactions effectuées entre le producteur, le transporteur et son concurrent le distributeur B. L'inverse est également vrai

Chaque nœud peut ensuite publier des transactions sur ses canaux respectifs depuis le CLI multichain à l'aide de la commande suivante :

```
publish stream1 '["key1","key2"]' '{"json":{"field1":"value"}}'
```

b. Modèle de transaction

Pour standardiser les transactions entre nœuds et ainsi faciliter le requêtage et le filtrage de la blockchain, nous proposons l'organisation suivante :

Les nœuds distributeur et producteurs auront accès à un format de transaction dit de « réception » qui permet de recenser de manière exhaustive le produit réceptionné (ou produit pour un producteur). Voici les champs que nous proposons pour organiser une transaction de la part de ces acteurs :


```
{
  "method": "publish",
  "params": [
    "org_blockchain",
    [
      "LOT:LOT-2025-QC-00421",
      "EXP:2025-07",
      "NODE:1FoodNode9xKpQrAbCdEfGhIj12345",
      "SKU:FROMAGE-BRIE-250G"
    ],
    {
      "json": {
        "schema_version": "1.0",
        "event_type": "RECEPTION",
        "timestamp_iso": "2025-04-19T14:32:00Z",
        "timestamp_unix": 1745070720,
        "lot_id": "LOT-2025-QC-00421",
        "sku": "FROMAGE-BRIE-250G",
        "product_name": "Brie de Meaux AOP 250g",
        "category": "PRODUITS_LAITIERS",
        "quantity_received": 48,
        "unit": "unites",
        "batch_number": "BATCH-FR-441-2025",
        "production_date": "2025-04-10",
        "expiry_date": "2025-07-15",
        "expiry_year_month": "2025-07",
        "producer_name": "Fromagerie Guilloteau",
        "producer_country": "FR",
        "certifications": ["AOP", "BIO"],
        "previous_node_id": "1SupplierNode3xLmNoPqRsTuVw67890",
        "receiving_node_address": "1FoodNode9xKpQrAbCdEfGhIj12345",
        "receiving_node_name": "Entrepôt Montréal-Est",
        "receiving_node_type": "ENTREPOT_REGIONAL",
        "gps_lat": 45.5831,
        "gps_lng": -73.5041,
        "temperature_reception_celsius": 4.2,
        "temperature_conforme": true,
        "packaging_intact": true,
        "conformity_check_passed": true,
        "inspector_address": "1InspectorXxYyZzAaBbCc11111",
        "transport_document_ref": "CMR-2025-04-19-FR-QC-0091",
        "recall_status": "NONE"
      }
```

```
    }
  ],
}
```

Le premier champ :

```
      "LOT:LOT-2025-QC-00421",
      "EXP:2025-07",
      "NODE:1FoodNode9xKpQrAbCdEfGhIj12345",
      "SKU:FROMAGE-BRIE-250G"
```

contient des clés qui agissent comme des métadonnées permettant de cibler spécifiquement des transactions. Ainsi, un utilisateur peut facilement filtrer les transactions de la blockchain selon si elles contiennent le `LOT`, la date d'expiration `EXP`, proviennent du nœud `NODE`, ou selon le code produit `SKU`. Ces options de de filtrage peuvent être combinées ensemble avec notamment le publisher (nœud ayant publié la transaction) pour créer des visualisations avancées par ordre chronologique, ou par flux de produit.

Par exemple avec la commande :

```
liststreamqueryitems stream1 '{"LOT":"LOT-
001","publisher":"1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM"}'
```

Un utilisateur peur rechercher tous les produits du lot `LOT-001` qui ont été traités par le nœud *`1WMEfuCevaq45RpfoQo6yVGoHF1qY6vaLBrKFM`*, correspondant ici au transporteur de l'organisation.

Les nœuds transporteurs auront eux accès uniquement à un format de transaction dit de « transfert » qui permet de recenser le produit transféré, et les parties prenantes de cet échange (source et destination) avec les mêmes possibilités de filtrage :

```
{
  "method": "publish",
  "params": [
    "lot-transfers",
    [
      "LOT:LOT-2025-QC-00421",
      "FROM:1FoodNode9xKpQrAbCdEfGhIj12345",
      "TO:1RetailNode7xMnOpQrStUv98765",
      "EXP:2025-07"
    ],
    {
      "json": {
        "schema_version": "1.0",
        "event_type": "TRANSFER",
        "timestamp_iso": "2025-04-22T09:00:00Z",
        "timestamp_unix": 1745312400,
        "lot_id": "LOT-2025-QC-00421",
        "quantity_transferred": 24,
        "unit": "unites",
        "from_node_address": "1FoodNode9xKpQrAbCdEfGhIj12345",
        "from_node_name": "Entrepôt Montréal-Est",
        "to_node_address": "1RetailNode7xMnOpQrStUv98765",
        "to_node_name": "IGA Plateau-Mont-Royal",
        "temperature_depart_celsius": 4.0,
        "temperature_arrivee_celsius": 3.8,
        "remaining_at_origin": 24
```

```
        }
      }
    ],}
```

Enfin, tous les nœuds auront accès à un dernier format de transaction dit « d'alerte » permettant de consigner dans la blockchain et signaler à tous les participants un rappel produit avec le lot concerné et la raison :

```
{
  "method": "publish",
  "params": [
    "lot-alerts",
    [
      "LOT:LOT-2025-QC-00421",
      "ALERT:RECALL",
      "EXP:2025-07",
      "SKU:FROMAGE-BRIE-250G"
    ],
    {
      "json": {
        "schema_version": "1.0",
        "event_type": "RECALL",
        "timestamp_iso": "2025-06-30T08:00:00Z",
        "lot_id": "LOT-2025-QC-00421",
        "alert_reason": "Contamination_Listeria",
        "severity": "CRITIQUE",
        "issued_by_authority": "MAPAQ",
        "reference_officielle": "MAPAQ-RECALL-2025-0831",
        "action_requise": "RETRAIT_IMMEDIAT",
        "lots_lies": ["LOT-2025-QC-00418", "LOT-2025-QC-00419"]
      }
    }
  ],
}
```

Une clé spécifique **`ALERT`**, permet de visualiser immédiatement toutes les alertes émises sur un canal.

c. Interactions avec la blockchain

Pour faciliter l'ajout des transactions standardisées précédentes, nous avons mis à disposition plusieurs scripts qui demandent interactivement les champs à remplir et ajoutent automatiquement les blocs à la blockchain. De surcroît, chaque nœud a à sa disposition un script « seed » permettant de remplir automatiquement la blockchain avec des transactions à but démonstratifs. Ces scripts se trouvent à l'emplacement `/root/chainscripts/`

Voici un récapitulatif des scripts à disposition :

Nom du script	*Utilité*	*Emplacement*
seed_distributeur_a.sh	Transactions de démo distributeur a	Nœud distributeur a
seed_distributeur_b.sh	Transactions de démo distributeur b	Nœud distributeur b
seed_producteur.sh	Transactions de démo producteur	Nœud producteur

seed_transporteur.sh	Transactions de démo transporteur	Nœud transporteur
publish_alert.sh	Publier une alerte sur produit	Tous les noeuds
publish_reception.sh	Publier une réception de produit	Nœuds distributeurs et producteur
publish_transfert.sh	Publier un transfert de produit	Nœud transporteur

Pour exécuter ces scripts vous devrez d'abord accorder la permission d'écriture **depuis l'hôte** (et non les conteneurs) dans le répertoire du projet à l'aide de la commande suivante :

```
nirin@LegIoN:~$ ls
compose.yaml  configs  dockerfile  setup_blockchain.sh
nirin@LegIoN:~$ chmod -R 755 configs/
```

Puis vous pourrez vous connecter au conteneur désiré pour exécuter les scripts de peuplement :

```
docker exec -it [NOM_IMAGE_CONTENEUR ou ID] bash
cd /root/chainscripts
./script_de_votre_choix.sh
```

Voici un exemple d'exécution du script de peuplement pour le nœud producteur :

```
root@e81a3015b8a6:~/chainscripts# ./seed_producteur.sh

[ CONNEXION BLOCKCHAIN ]

  Nom de la blockchain        (ex: org_blockchain)        : org_blockchain
  Stream des réceptions       (ex: streamA)               : streamA

==============================================
  SEED — PRODUCTEUR : Ferme Avicole Ste-Marie
==============================================
  Blockchain  : org_blockchain
  Lot         : LOT-2025-QC-00421
  Produit     : Poulet entier frais Label Rouge 1.5 kg
  Quantité    : 200 unites
  Abattage    : 2025-04-09
  Péremption  : 2025-04-17
==============================================

  Événements qui seront publiés :
    1. RECEPTION  — création du lot chez le producteur

  Note : le producteur ne publie PAS de transfert.
         Il recense uniquement les lots qu'il crée.

  Lancer le seed ? (oui/non) : oui

  [1/1] Publication de la RÉCEPTION...
{"method":"publish","params":["streamA",["LOT:LOT-2025-QC-00421","EXP:2025-04","NODE:1ProducerNodeAaBbCcDdEeFfGg11111","SKU:POULET-ENTIER-FRAIS-1KG5"],{"json":{"schema
_version":"1.0","event_type":"reception","timestamp_iso":"2025-04-10T06:00:00Z","lot_id":"LOT-2025-QC-00421","sku":"POULET-ENTIER-FRAIS-1KG5","product_name":"Poulet en
tier frais Label Rouge 1.5 kg","category":"volaille_fraiche","batch_number":"BATCH-QC-AVI-2025-0421","production_date":"2025-04-10","abattage_date":"2025-04-09","expir
y_date":"2025-04-17","expiry_year_month":"2025-04","quantity_received":200,"unit":"unites","producer_name":"Ferme Avicole Ste-Marie","producer_country":"ca","producer_
region":"Chaudi\u00E8re-Appalaches","certifications":["label_rouge","bien_etre_animal"],"previous_node_id":null,"receiving_node_address":"1ProducerNodeAaBbCcDdEeFfGg11
111","receiving_node_name":"Ferme Avicole Ste-Marie","receiving_node_type":"producteur","gps_lat":46.4471,"gps_lng":-71.0381,"operator":"Ferme Avicole Ste-Marie","temp
erature_reception_celsius":2,"temperature_conforme":true,"packaging_intact":true,"conformity_check_passed":true,"inspector_address":"1ProducerNodeAaBbCcDdEeFfGg11111",
"transport_document_ref":null,"customs_ref":null,"recall_status":"none"}},"offchain"],"id":"77608380-1776974649","chain_name":"org_blockchain"}

      ✓ RECEPTION publiée — TXID : 7f67b611bc04630f03c8c04fb09e9cf76e9ac9180499ad6637c06732aa64fb75

==============================================
  SEED PRODUCTEUR TERMINÉ
  1 événement publié sur org_blockchain
==============================================
```

d. Visualisation et filtrage

Pour rappel, vous devez d'abord entrer dans l'interface multichain-cli à l'aide de la commande suivante pour exécuter les commandes natives à multichain qui suivront :

```
multichain-cli org_blockchain
```

Une fois toutes les transactions ajoutées désirées, vous pouvez visualiser toutes les transactions d'un canal à partir d'un conteneur l'aide de l'outil multichain-cli : `liststreamitems [STREAM]`

```
nirin@LegIoN:~$ docker exec nirin-producteur-node-1 multichain-cli org_blockchain liststreamitems streamA
{"method":"liststreamitems","params":["streamA"],"id":"36270664-1776976138","chain_name":"org_blockchain"}

[
    {
        "publishers" : [
            "1YpuJ2hciVX3q5fmoWrnihYkouN1HjbL5hnSXg"
        ],
        "keys" : [
            "LOT:LOT-2025-QC-00421",
            "EXP:2025-04",
            "NODE:1ProducerNodeAaBbCcDdEeFfGg11111",
            "SKU:POULET-ENTIER-FRAIS-1KG5"
        ],
        "offchain" : true,
        "available" : true,
        "data" : {
            "json" : {
                "schema_version" : "1.0",
                "event_type" : "reception",
                "timestamp_iso" : "2025-04-10T06:00:00Z",
                "lot_id" : "LOT-2025-QC-00421",
                "sku" : "POULET-ENTIER-FRAIS-1KG5",
                "product_name" : "Poulet entier frais Label Rouge 1.5 kg",
                "category" : "volaille_fraiche",
                "batch_number" : "BATCH-QC-AVI-2025-0421",
                "production_date" : "2025-04-10",
                "abattage_date" : "2025-04-09",
                "expiry_date" : "2025-04-17",
                "expiry_year_month" : "2025-04",
                "quantity_received" : 200,
                "unit" : "unites",
                "producer_name" : "Ferme Avicole Ste-Marie",
                "producer_country" : "ca",
                "producer_region" : "Chaudi\u00E8re-Appalaches",
                "certifications" : [
                    "label_rouge",
                    "bien_etre_animal"
```

...SNAP...

Vous pouvez également visualiser les transactions enregistrées sur la blockchain par un nœud particulier, en utilisant son identifiant hash multichain : `liststreamqueryitems [CANAL] '{"publisher":"HASH_MUTLICHAIN_NOEUD"}'`

```
org_blockchain: liststreamqueryitems streamB '{"publisher":"17iJKokEbNj3E3nrk2A5hY5WdKTY3GTLpwG9sZ"}'
{"method":"liststreamqueryitems","params":["streamB",{"publisher":"17iJKokEbNj3E3nrk2A5hY5WdKTY3GTLpwG9sZ"}],"id":"24911891-1776977274","chain_name":"org_blockchain"}

[
    {
        "publishers" : [
            "17iJKokEbNj3E3nrk2A5hY5WdKTY3GTLpwG9sZ"
        ],
        "keys" : [
            "LOT:LOT-2025-QC-00421",
            "FROM:1TransportNodeHhIiJjKkLlMm22222",
            "TO:1DistribANodeNnOoPpQqRrSs33333",
            "EXP:2025-04"
        ],
        "offchain" : true,
        "available" : true,
        "data" : {
            "json" : {
                "schema_version" : "1.0",
                "event_type" : "transfer",
                "timestamp_iso" : "2025-04-10T16:00:00Z",
                "lot_id" : "LOT-2025-QC-00421",
                "expiry_date" : "2025-04-17",
                "expiry_year_month" : "2025-04",
                "transfer_date" : "2025-04-10",
                "quantity_transferred" : 100,
                "unit" : "unites",
                "remaining_at_origin" : 100,
                "from_node_address" : "1TransportNodeHhIiJjKkLlMm22222",
                "from_node_name" : "Transport R\u00E9frig\u00E9r\u00E9 Laurentides",
                "from_node_type" : "transporteur",
                "to_node_address" : "1DistribANodeNnOoPpQqRrSs33333",
                "to_node_name" : "Boucherie Gros-Volume Laval",
                "to_node_type" : "point_de_vente",
                "temperature_depart_celsius" : 2.2,
                "temperature_arrivee_celsius" : 2.5,
                "temperature_conforme" : true,
                "transport_document_ref" : "BL-2025-04-10-QC-0071",
                "sender_address" : "1TransportNodeHhIiJjKkLlMm22222",
                "receiver_address" : "1DistribANodeNnOoPpQqRrSs33333"
            }
        },
```

...SNAP...

Enfin vous pouvez cibler des transactions spécifiques, comme les alertes (contamination, péremption, etc) pour faciliter l'identification des lots à traiter, et les parties prenantes : `liststreamqueryitems streamA '{"keys":["ALERT:EXPIRY_WARNING"]}' # Exemples de valeurs possibles: ["ALERT:QUALITY_HOLD","ALERT:CONTAMINATION", "ALERT:RECALL"]`

```
org_blockchain: liststreamqueryitems streamA '{"keys":["ALERT:EXPIRY_WARNING"]}'
{"method":"liststreamqueryitems","params":["streamA",{"keys":["ALERT:EXPIRY_WARNING"]}],"id":"35363551-1776977812","chain_name":"org_blockchain"}

[
    {
        "publishers" : [
            "1Uuu47HVvxSDLnBkB1iqEsQ5rrDxNDh12CHd2Z"
        ],
        "keys" : [
            "LOT:LOT-2025-QC-00421",
            "ALERT:EXPIRY_WARNING",
            "EXP:2025-04",
            "SKU:POULET-ENTIER-FRAIS-1KG5"
        ],
        "offchain" : true,
        "available" : true,
        "data" : {
            "json" : {
                "schema_version" : "1.0",
                "event_type" : "expiry_warning",
                "timestamp_iso" : "2025-04-16T07:30:00Z",
                "lot_id" : "LOT-2025-QC-00421",
                "sku" : "POULET-ENTIER-FRAIS-1KG5",
                "expiry_date" : "2025-04-17",
                "expiry_year_month" : "2025-04",
                "detection_date" : "2025-04-16",
                "alert_type" : "expiry_warning",
                "alert_reason" : "volaille_fraiche_perimant_dans_24h",
                "severity" : "majeure",
                "action_requise" : "retrait_rayon_et_destruction",
                "issued_by_authority" : "interne",
                "reference_officielle" : "INTERNAL-BGV-LAV-2025-0416-001",
                "issuer_node_address" : "1DistribANodeNnOoPpQqRrSs33333",
                "lots_lies" : [
                ],
                "nodes_concernes" : [
                    "1DistribANodeNnOoPpQqRrSs33333"
                ],
                "report_url" : null,
                "commentaire" : "Controle rayon quotidien \u2014 18 unites encore en stock a retirer immediatement"
            }
        },
        "confirmations" : 45,
        "blocktime" : 1776974948,
        "txid" : "488a99d2d7d298ba4fed521f71fa401c108d9b941f67ac9f31e39595ebae78ab"
    },
    {
        "publishers" : [
```